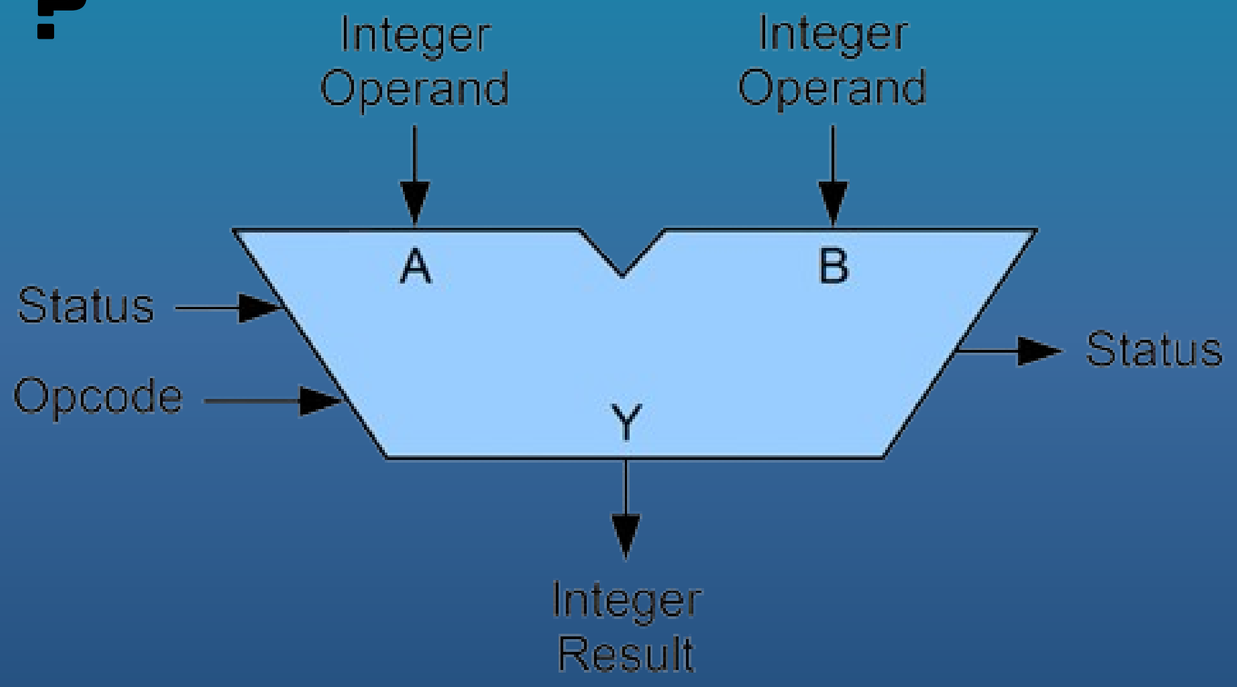
8-BIT ALU SIMULATION

- IORDACONIU ALEXANDRA
- JDIOREANTU DEALIA
- LEAHU ANDREEA

CE ESTE UN ALU?

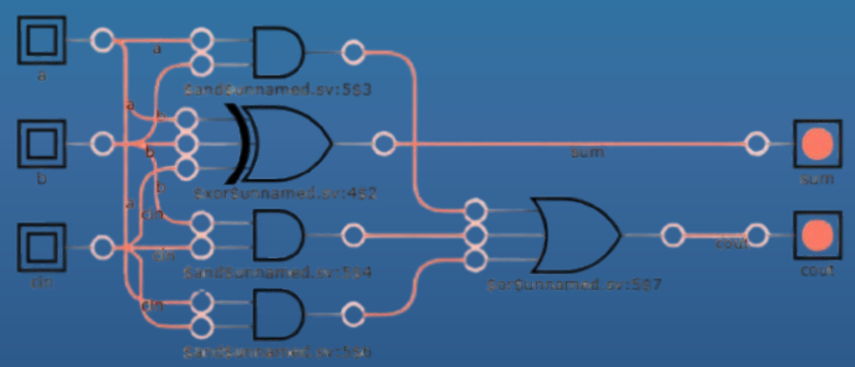
ALU (Unitatea Aritmetică și Logică) este o parte fundamentală a procesorului, care realizează operații aritmetice (adunare, scădere, înmulțire, împărțire) și logice (AND, OR, XOR, NOT) asupra datelor binare, având un rol direct în procesarea instrucțiunilor și în luarea deciziilor într-un sistem digital.



ARHITECTURĂ MODULARĂ — DECIZIE DE PROIECTARE

Pentru implementarea ALU-ului, am ales o arhitectură modulară, în care fiecare operație (adunare, scădere, înmulțire și împărțire) este realizată într-un modul separat. Această organizare asigură o structură clară, permite reutilizarea și testarea facilă a fiecărei componente și oferă posibilitatea optimizării independente a operațiilor (de exemplu, Booth radix-2 pentru înmulțire și non-restoring pentru împărțire). Rezultatul final este selectat în funcție de semnalul de control (op_code), printr-un multiplexor plasat la ieșirea ALU-ului.

ADUNARE
RIPPLE CARRY ADDER



Adunarea este realizată folosind o structură de tip Ripple Carry Adder pe 8 biți, formată din mai multe blocuri Full Adder conectate în serie. Fiecare bit este procesat succesiv, iar semnalul de carry se propagă de la un etaj la următorul, asigurând corectitudinea rezultatului. Pentru această operație, operandul B este utilizat direct, iar carry-in-ul inițial este 0.

SCĂDERE
COMPLEMENT FAȚĂ DE 2

Scăderea este implementată folosind metoda complementului față de 2, reutilizând aceeași structură de adunare. Operandul B este mai întâi complementat (~B), iar carry-in-ul inițial este setat la 1, obținându-se astfel operația A - B sub forma A + (~B) + 1. Selectarea între B și ~B se realizează cu ajutorul unui multiplexor pe 8 biți, controlat de semnalul sub.

ÎNMULȚIRE ALGORITMUL BOOTH RADIX-2

Înmulțirea a fost implementată folosind algoritmul Booth Radix-2, pentru a realiza operația de multiplicare eficient în reprezentare în complement față de 2. În această implementare:

Analiza perechilor de biți: Algoritmul examinează biții consecutivi ai multiplicatorului (Q0 și Q-1) pentru a decide operația aritmetică necesară asupra registrului A

. Operații aritmetice condiționate: În funcție de combinația de biți, se efectuează

adunare (A + M)
scădere (A - M)
nicio operație

optimizând astfel numărul de calcule necesare.

Shift aritmetic: După fiecare etapă, registrele A și Q sunt deplasate aritmetic la dreapta, iar bitul Q-1 este actualizat pentru a păstra informația de control a pașilor anteriori. Sut prin concatenarea registrelor A și Q, reprezentând produsul pe 16 biți în format semnat.

ÎMPĂRȚIRE
ALGORITMUL NON-RISTORING

Împărțirea a fost implementată folosind algoritmul Non-Restoring Division, pentru a realiza operația de divizare eficientă în reprezentare în complement față de 2. În această implementare:

Structură iterativă pe pași: Algoritmul parcurge împărțirea în mai multe cicluri, în care registrul de rest (A) și registrul de cât (Q) sunt actualizate progresiv.

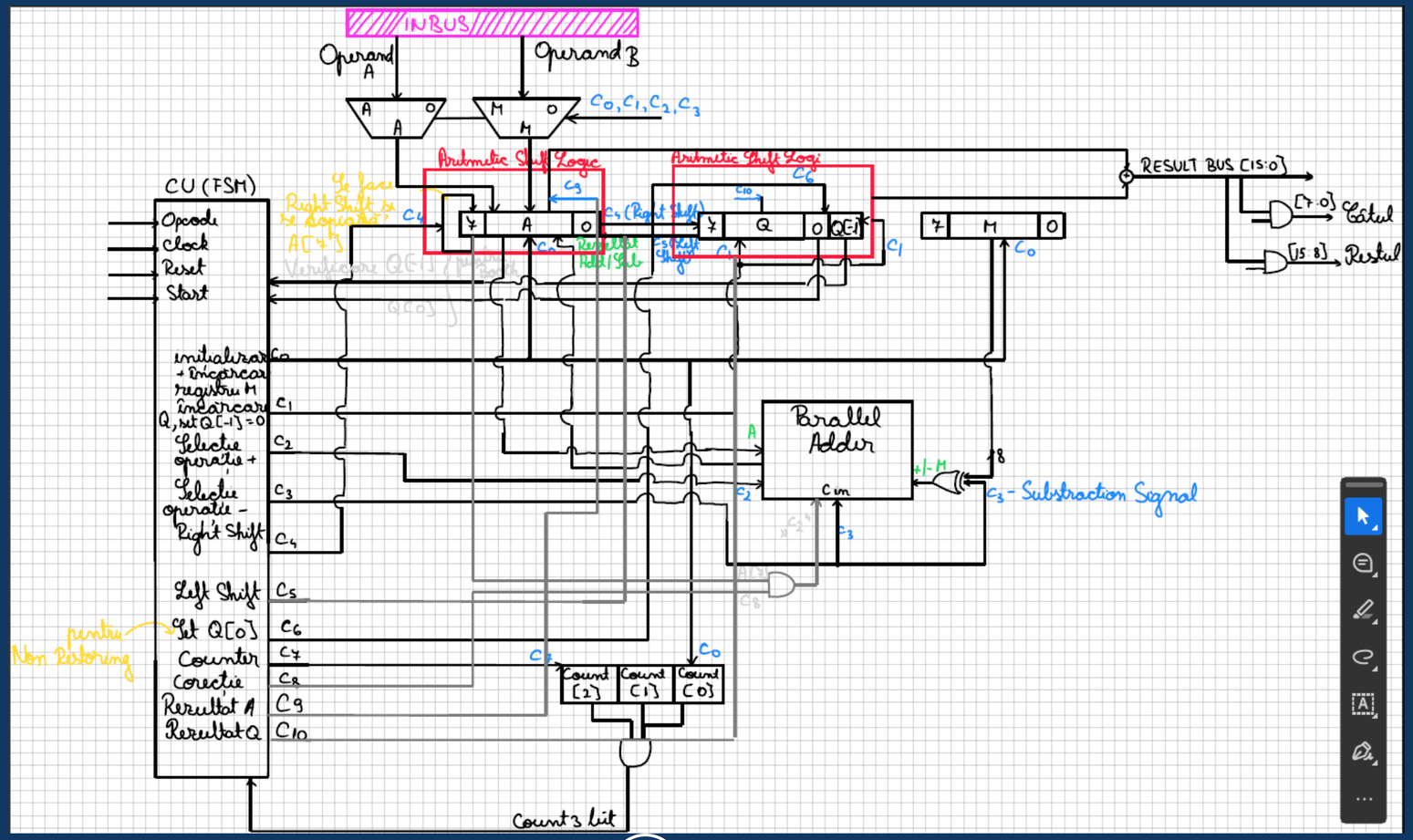
Operații condiționate pe semnul restului: În funcție de semnul registrului A (bitul cel mai semnificativ), se decide operația aritmetică:

- dacă A este negativ → se adaugă divisorul (A + M)
- dacă A este pozitiv → se scade divisorul (A - M)

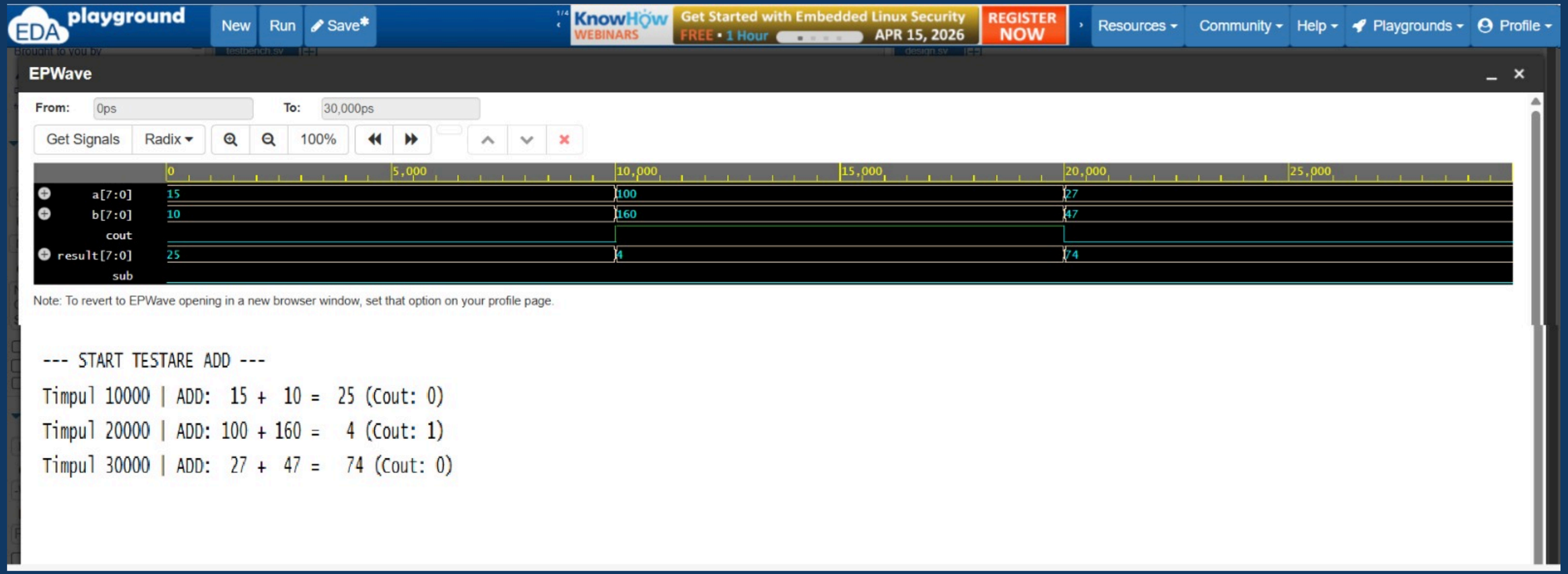
Shift aritmetic combinat: După fiecare pas, registrele A și Q sunt deplasate la stânga, iar bitul cel mai semnificativ din Q este completat în funcție de rezultatul operației curente, menținând fluxul algoritmului.

La final, dacă restul (A) este negativ, se aplică o corecție finală prin adunarea divisorului. Rezultatul final este format din Q (câtul) și A (restul), reprezentând rezultatul complet al împărțirii.

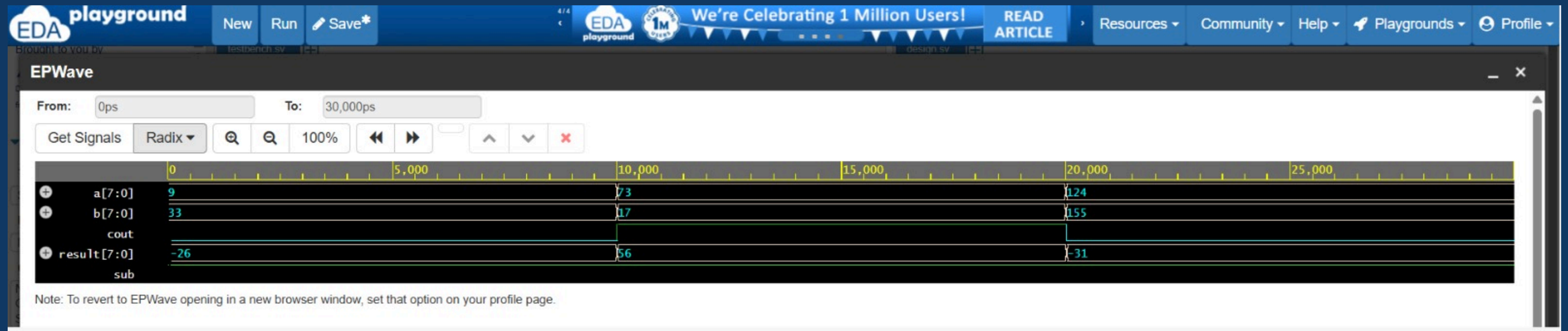
DIAGRAMA HARDWARE



WAVE ADDER

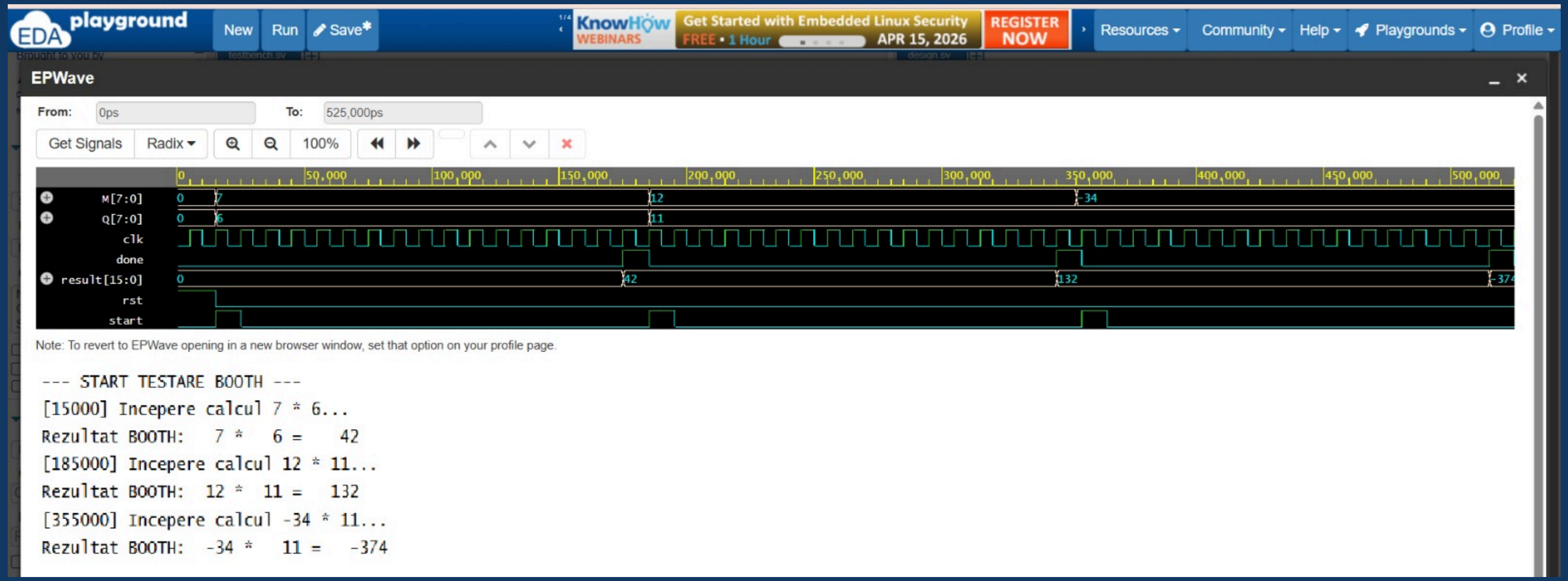

EDA playground
New
Run
Save*
KnowHow WEBINARS
Get Started with Embedded Linux Security
FREE • 1 Hour
APR 15, 2026
REGISTER NOW
Resources
Community
Help
Playgrounds
Profile
EPWave
From: 0ps
To: 30,000ps
Get Signals
Radix
100%
0
5,000
10,000
15,000
20,000
25,000
a[7:0]
b[7:0]
cout
result[7:0]
sub
15
10
25
100
160
4
27
47
74
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.
--- START TESTARE ADD ---
Timpul 10000 | ADD: 15 + 10 = 25 (Cout: 0)
Timpul 20000 | ADD: 100 + 160 = 4 (Cout: 1)
Timpul 30000 | ADD: 27 + 47 = 74 (Cout: 0)

WAVE SUBSTRACTION

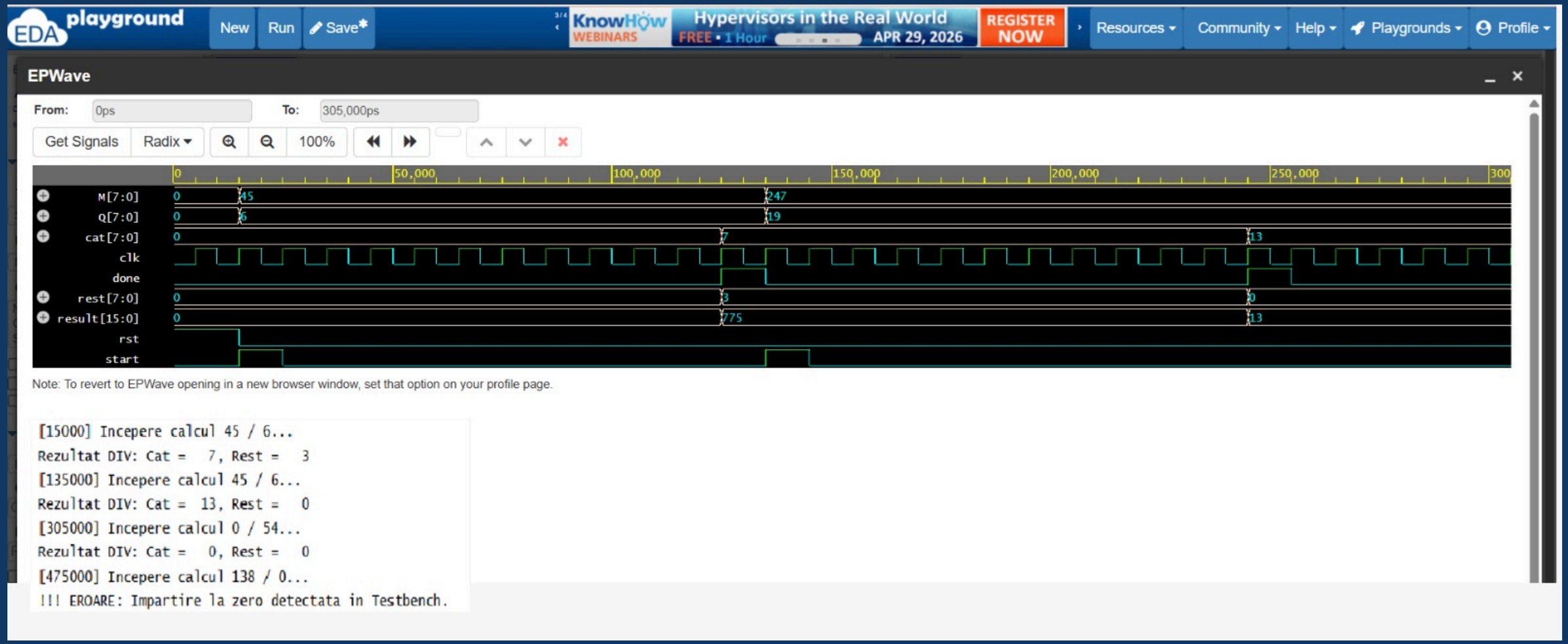



```
--- START TESTARE SUB ---
Timpul 10000 | SUB:  81 - 107 =  -26 (Cout: 0)
Timpul 20000 | SUB:  73 -  17 =   56 (Cout: 1)
Timpul 30000 | SUB: 124 - 155 =  -31 (Cout: 0)
```

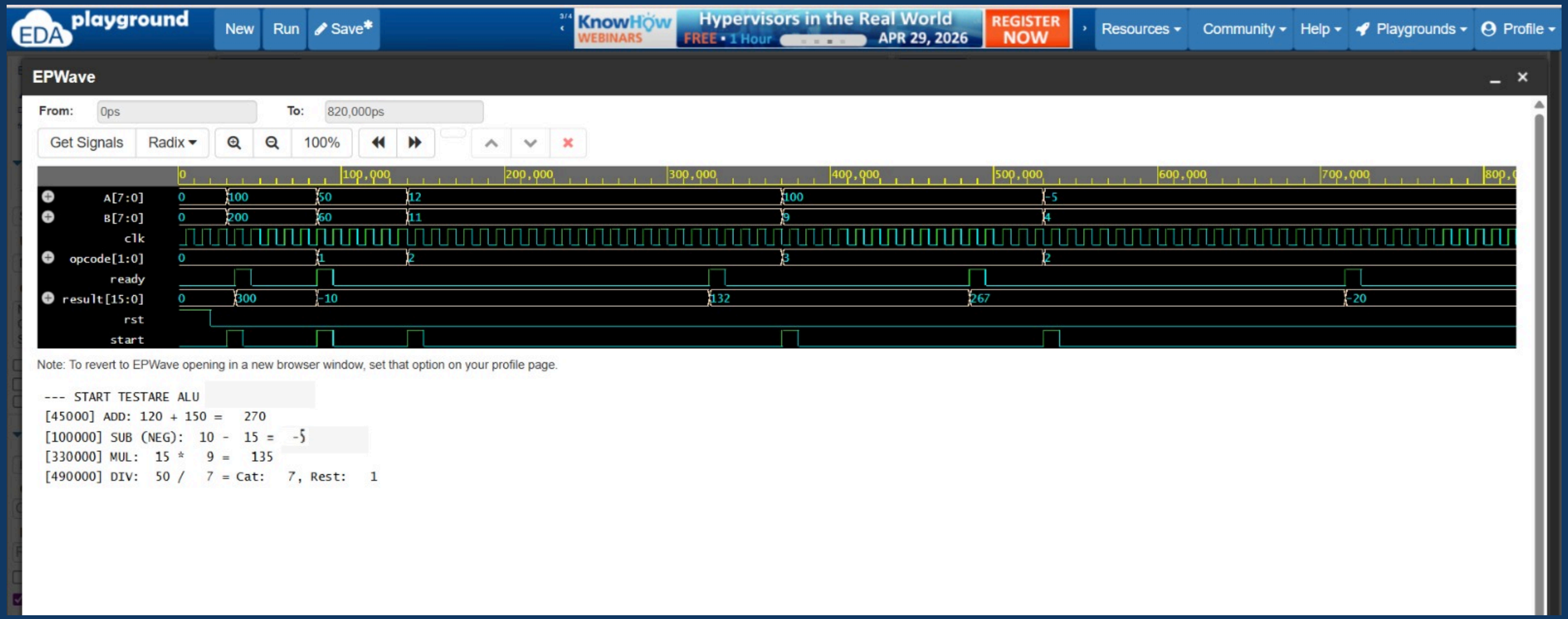
WAVE IMPARTIRE



WAVE IMPARTIRE



WAVE MAIN


EDA playground
New
Run
Save*
KnowHow WEBINARS
Hypervisors in the Real World
FREE • 1 Hour
APR 29, 2026
REGISTER NOW
Resources
Community
Help
Playgrounds
Profile
EPWave
From: 0ps
To: 820,000ps
Get Signals
Radix
100%
A[7:0]
B[7:0]
clk
opcode[1:0]
ready
result[15:0]
rst
start
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.
--- START TESTARE ALU
[45000] ADD: 120 + 150 = 270
[100000] SUB (NEG): 10 - 15 = -5
[330000] MUL: 15 * 9 = 135
[490000] DIV: 50 / 7 = Cat: 7, Rest: 1

MULȚUMIM!